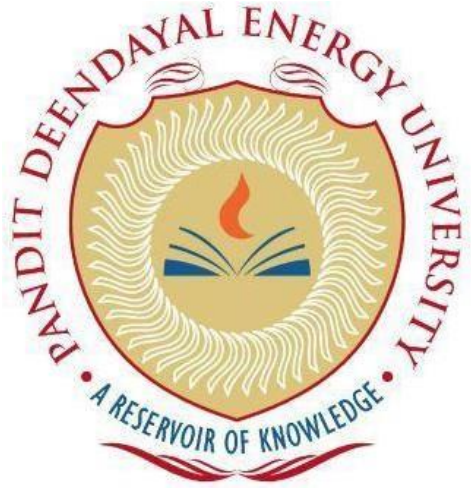
PANDIT DEENDAYAL ENERGY UNIVERSITY

SCHOOL OF TECHNOLOGY



Course: Machine Learning Lab

Course Code: 20CP401P

LAB MANUAL

Submitted To:

Dr. Debabrata Swain

Submitted By:

Pearl Kalariya
23BCP248

Experiment 3

Title:

Implement simple and multi-linear regression to predict profits for a food truck. Compare the performance of the model on linear and multi-linear regression.

Objective:

The objective of this lab assignment is to implement simple and multi-linear regression models to predict profits for a food truck business. By comparing the performance of these two regression models, you will gain insights into when and how to use simple and multi-linear regression techniques.

Dataset Format:

Population	Years in business	Profit
10,000	5	10000
15000	6	12000
20000	6	13000
9000	5	12000
12000	4	?

Tasks:

1. Apply Simple Linear Regression Selecting.
2. Performance Evaluation (Simple Linear Regression).
3. Multi-Linear Regression.
4. Performance Evaluation (Multi-Linear Regression).
5. Model Comparison and Interpretation

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

plt.style.use('seaborn-v0_8-darkgrid')

data = {
    'Population': [10000, 15000, 20000, 8000, 12000],
    'YearsInBusiness': [5, 6, 6, 5, 4],
    'Profit': [10000, 12000, 13000, 12000, np.nan]
}

df = pd.DataFrame(data)
df

known_df = df.dropna().reset_index(drop=True)
unknown_df = df[df['Profit'].isna()].reset_index(drop=True)

known_df
```

Output:

	# Population	# YearsInBusiness	# Profit
0	10000	5	10000.0
1	15000	6	12000.0
2	20000	6	13000.0
3	8000	5	12000.0
4	12000	4	*Missing value*

	# Population	# YearsInBusiness	# Profit
0	10000	5	10000.0
1	15000	6	12000.0
2	20000	6	13000.0
3	8000	5	12000.0

Task 1: Apply Simple Linear Regression

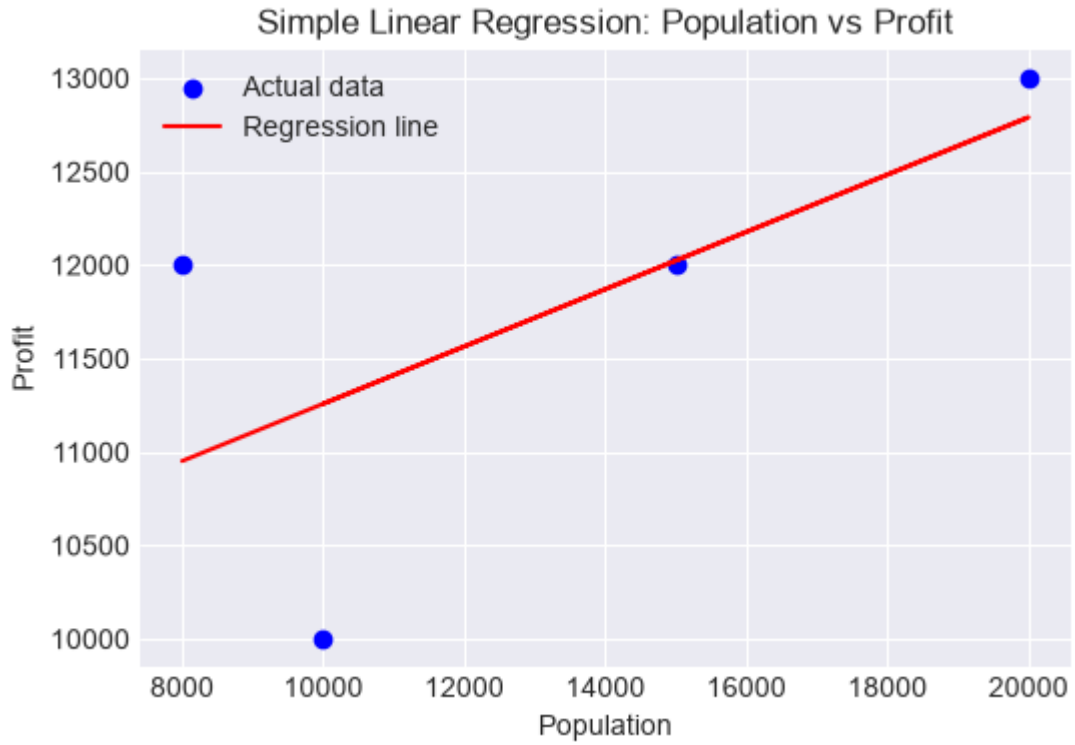
Code:

```
X = known_df[['Population']]
y = known_df['Profit']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
random_state=42)

simple_model = LinearRegression()
simple_model.fit(X_train, y_train)

print('Intercept:', simple_model.intercept_)
print('Coefficient:', simple_model.coef_)

y_pred_simple = simple_model.predict(X_test)

pd.DataFrame({'Actual': y_test.values, 'Predicted': y_pred_simple})

plt.figure(figsize=(6,4))
plt.scatter(known_df['Population'], known_df['Profit'], color='blue',
label='Actual data')
plt.plot(known_df['Population'],
simple_model.predict(known_df[['Population']]), color='red', label='Regression
line')
plt.xlabel('Population')
plt.ylabel('Profit')
plt.title('Simple Linear Regression: Population vs Profit')
plt.legend()
plt.show()
```

Output:

```
Intercept: 9725.806451612903
Coefficient: [0.15322581]
```

	# Actual	# Predicted
0	12000.0	12024.193548387097



Task 2: Performance Evaluation (Simple Linear Regression)

Code:

```
simple_mae = mean_absolute_error(y_test, y_pred_simple)
simple_mse = mean_squared_error(y_test, y_pred_simple)
simple_rmse = np.sqrt(simple_mse)
simple_r2 = r2_score(y_test, y_pred_simple)

print('MAE:', simple_mae)
print('MSE:', simple_mse)
print('RMSE:', simple_rmse)
print('R2 Score:', simple_r2)
```

Output:

```
MAE: 24.193548387096598
MSE: 585.3277835587844
RMSE: 24.193548387096598
R2 Score: nan
```

Task 3: Multi-Linear Regression

Code:

```
X_multi = known_df[['Population', 'YearsInBusiness']]
y_multi = known_df['Profit']

X_train_m, X_test_m, y_train_m, y_test_m = train_test_split(X_multi, y_multi,
test_size=0.25, random_state=42)

multi_model = LinearRegression()
multi_model.fit(X_train_m, y_train_m)

print('Intercept:', multi_model.intercept_)
print('Coefficients:', multi_model.coef_)

y_pred_multi = multi_model.predict(X_test_m)

pd.DataFrame({'Actual': y_test_m.values, 'Predicted': y_pred_multi})
```

Output:

```
Intercept: -45000.000000000044
Coefficients: [-1.0e+00  1.3e+04]
```

	# Actual	# Predicted
0	12000.0	18000.000000000007

Task 4: Performance Evaluation (Multi-Linear Regression)

Code:

```
multi_mae = mean_absolute_error(y_test_m, y_pred_multi)
multi_mse = mean_squared_error(y_test_m, y_pred_multi)
multi_rmse = np.sqrt(multi_mse)
multi_r2 = r2_score(y_test_m, y_pred_multi)

print('MAE:', multi_mae)
print('MSE:', multi_mse)
print('RMSE:', multi_rmse)
print('R2 Score:', multi_r2)
```

Output:

```
MAE: 6000.000000000007
MSE: 36000000.00000009
RMSE: 6000.000000000007
R2 Score: nan
```

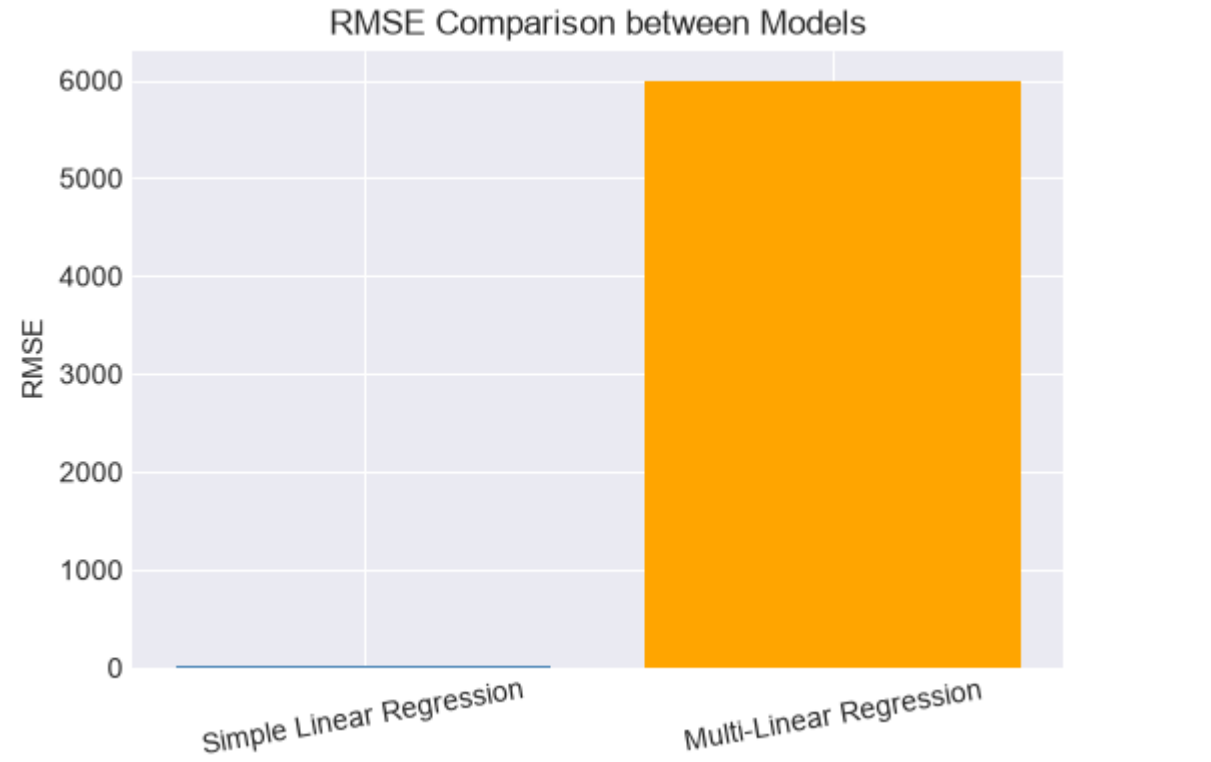
Task 5: Model Comparison and Interpretation

Code:

```
comparison = pd.DataFrame({
    'Model': ['Simple Linear Regression', 'Multi-Linear Regression'],
    'MAE': [simple_mae, multi_mae],
    'MSE': [simple_mse, multi_mse],
    'RMSE': [simple_rmse, multi_rmse],
    'R2 Score': [simple_r2, multi_r2]
})

Comparison

fig, ax = plt.subplots(figsize=(6,4))
ax.bar(comparison['Model'], comparison['RMSE'], color=['steelblue', 'orange'])
ax.set_ylabel('RMSE')
ax.set_title('RMSE Comparison between Models')
plt.xticks(rotation=10)
plt.show()

predicted_profit_simple = simple_model.predict(unknown_df[['Population']])
predicted_profit_multi = multi_model.predict(unknown_df[['Population',
'YearsInBusiness']])

print('Predicted profit (Simple Linear Regression):',
predicted_profit_simple[0])
print('Predicted profit (Multi-Linear Regression):',
predicted_profit_multi[0])
```

Output:

	Model	# MAE	# MSE	# RMSE	# R2 Score
0	Simple Linear Regression	24.193548387096598	585.3277835587844	24.193548387096598	*Missing value*
1	Multi-Linear Regression	6000.000000000007	36000000.00000009	6000.000000000007	*Missing value*




```
Predicted profit (Simple Linear Regression): 11564.516129032258
Predicted profit (Multi-Linear Regression): -5000.000000000015
```